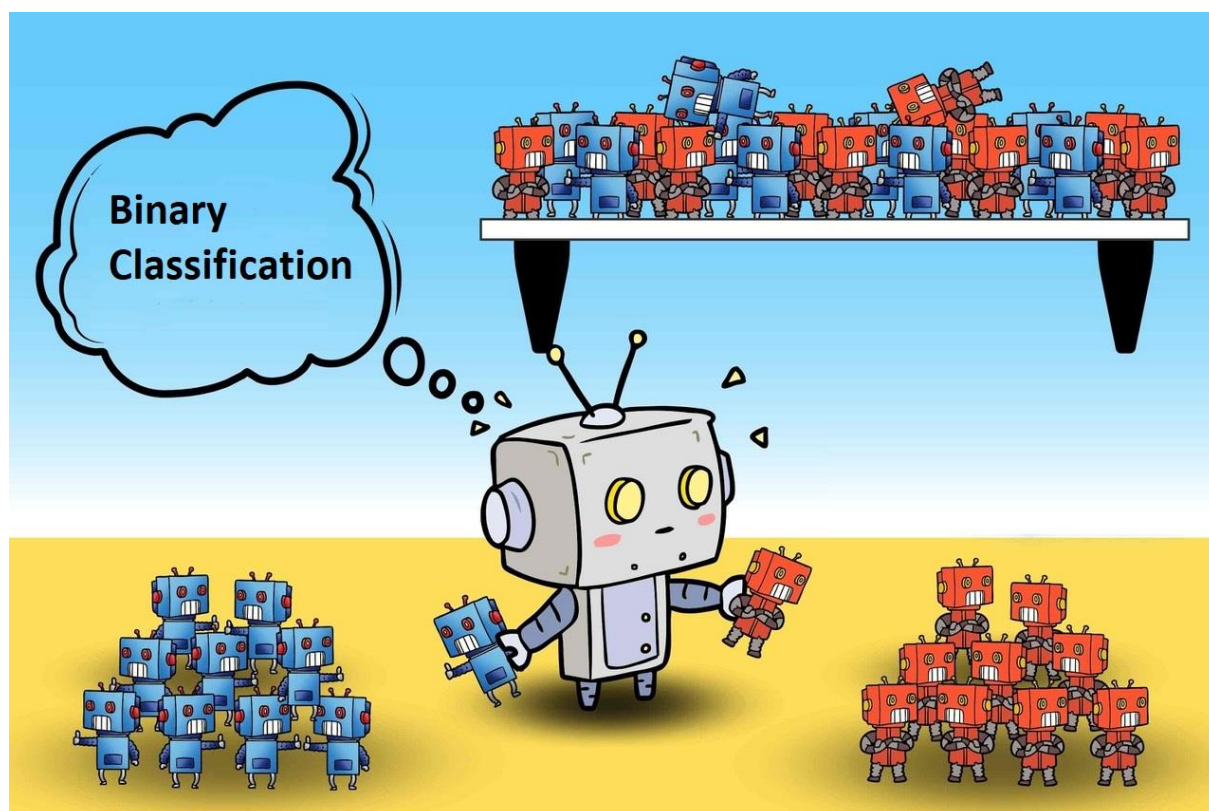


Livrable - E3



Franck Le Fur

Le 1^{er} avril 2026

Titre professionnel **Développeur en intelligence artificielle**
de niveau 6 enregistré au RNCP sous le n°37827.

Passage par la voie de la formation - parcours de 19 mois achevé en mars 2026

Table des matières

Présentation	3
Présentation du projet.....	3
Spécifications techniques	3
Architecture du projet.....	4
Préparation des données	4
Collecte des données	4
Nettoyage des textes.....	5
Chunking.....	5
Le modèle d'IA [C11, C12].....	6
Vectorisation des textes	6
Annoter les documents	7
Modèle : régression logistique.....	7
Séparation des données en données d'entraînement et en données de tests	7
Entraînement du modèle	8
Les métriques d'un modèle de classification	8
Evaluation de notre modèle	9
Monitoring du modèle avec MLFlow [C11].....	10
Tests automatisés d'un modèle d'IA [C12]	11
Architecture d'API [C9, C10]	13
API FastAPI exposant le modèle d'IA [C9].....	13
Règles d'authentification.....	14
Exemples d'utilisation de l'API	15
API Flask exposant l'application [C10]	15
Utilisation de l'API Flask	16
Déploiement sur Render [C13].....	17
Conclusion	18
Annexes.....	20

Présentation

Présentation du projet

Pendant mon alternance au Crédit Agricole, mon projet principal a été de développer un **moteur de recherche documentaire** au sein du service « **Assurances** ».

Techniquement ce moteur de recherche s'appuie sur des outils d'extraction de textes, de nettoyage de texte (**NLP, Spacy**), de vectorisation de textes (**TF-IDF, Embedding**), de création d'une application Web utilisateur (**Streamlit**) puis des méthodes de calcul de **similarité sémantique** pour trouver les documents pertinents en fonction d'une saisie utilisateur.

Les contraintes techniques inhérentes au Crédit Agricole et les besoins du projet font qu'il ne contient **ni API ni entraînement d'un modèle d'IA**. Parallèlement à ce projet de société, j'ai donc développé un modèle d'IA pour réaliser une **classification automatique de textes** dans le domaine des assurances.

L'**objectif** de ce modèle d'IA est de déterminer si un mot, une phrase ou une courte requête saisie par un utilisateur **appartient au domaine des assurances ou non** et d'utiliser ce modèle classifieur comme **un filtre** du moteur de recherche avant de renvoyer une liste de résultats. Ce filtre détermine si la saisie utilisateur **concerne bien le domaine des assurances**, dans ce cas le moteur de recherche renvoie une liste de documents pertinents, dans le cas contraire le moteur de recherche affiche simplement un message pour dire que la saisie utilisateur ne semble pas concerner les assurances et invite l'utilisateur à reformuler sa demande.

Ce projet permet d'entraîner une IA, de la monitorer, de la mettre sous forme d'API et de l'inclure dans une application, elle-même sous forme d'une API.

Spécifications techniques

Ce projet sera intégralement mené en langage **Python**, ce langage offre gratuitement toutes les librairies nécessaires à la conduite de ce projet.

Manipulation, nettoyage et agrégation des données seront faits à l'aide des librairies **Panda, pdfplumber, Scipy, Pickle**. La vectorization des données sera faite à l'aide de **Scikit-Learn**. L'API exposant le modèle d'IA sera créée avec **FastAPI**, sa documentation sera faite selon le modèle **OpenAPI**, enfin et l'applications Web sera développée à l'aide de **Flask**.

Le dépôt Github est consultable ici : https://github.com/Franck-LF/Projet_CA

Architecture du projet

La figure suivante présente le pipeline de traitement mis en place pour la classification des textes.

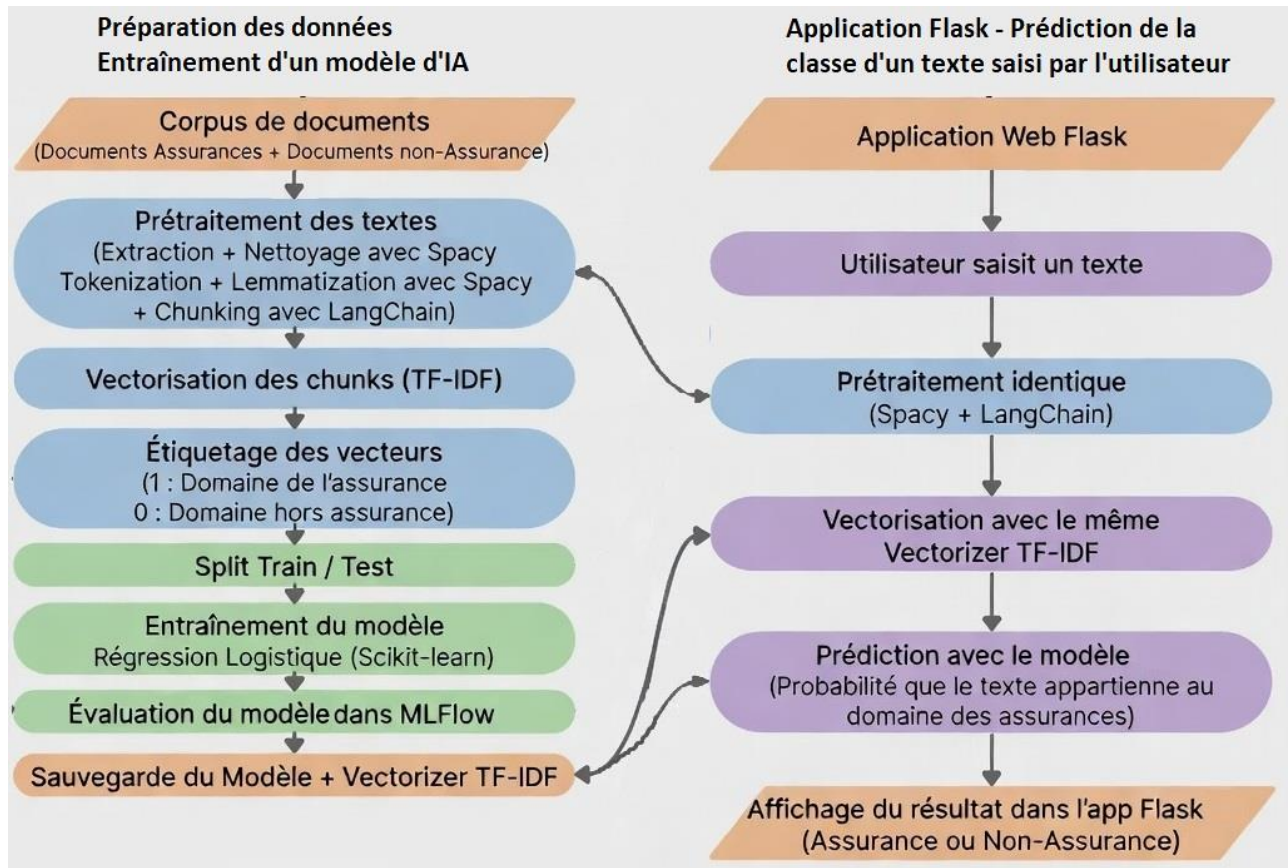


Figure 1 - Schéma du projet Classification Documentaire

Préparation des données

Collecte des données

La première étape du projet a consisté à **constituer un corpus de données textuelles**. Pour cela, deux catégories de documents ont été collectées : d'une part **des documents liés au domaine de l'assurance** et d'autre part des documents appartenant à des domaines hors assurance (sport, sciences, cuisine, informatique ...), l'ensemble de ces documents sont publics et ont été collecté sur internet. Cette séparation permet de construire un jeu de données adapté à l'entraînement d'un modèle de classification binaire. L'objectif est de fournir au modèle des exemples

représentatifs des deux classes afin qu'il puisse apprendre à distinguer les caractéristiques linguistiques associées aux deux classes.

Nettoyage des textes

Cette étape consiste à extraire le texte brut des documents à l'aide de la librairie **pdfplumber**, puis à nettoyer le texte brut :

- Passage en minuscules
- Retirer les caractères spéciaux
- Supprimer les liens et chemins
- Supprimer les retours à la ligne et les espaces inutiles

Ensuite à l'aide de la librairie **Spacy** et du modèle français « **fr_core_news_md** », on effectue une **tokenization** des textes, étape pendant laquelle on supprime les **ponctuations** et les **stop-words** français, finalement on effectue une **lemmatization** des documents, c'est-à-dire que l'on garde uniquement la racine des mots (on retire les marques de pluriel, de genre ou les terminaisons de conjugaison).

Chunking

Cette étape consiste à découper les textes nettoyés en unités plus homogènes, facilitant ainsi l'apprentissage du modèle.

Puisque ce modèle d'IA a pour finalité de servir de filtre à notre moteur de recherche on se place dans la situation utilisateur du moteur de recherche où l'on peut raisonnablement considérer que les utilisateurs saisiront entre 2 et 8 mots.

Nous allons donc découper nos textes en unités comportant entre 2 et 8 mots, par exemple la phrase « **La simulation de crédit intégrant les caractéristiques de votre projet de financement.** » génère les chunks suivants :

```
['intégrer credit', 'credit simulation', 'intégrer projet credit caractéristique', 'intégrer c  
redit simulation', 'intégrer credit caractéristique', 'projet financement caractéristique', 'i  
ntégrer credit simulation caractéristique', 'intégrer projet caractéristique', 'intégrer proje  
t credit financement caractéristique', 'intégrer projet financement credit caractéristique sim  
ulation', 'intégrer caractéristique', 'projet financement', 'projet caractéristique', 'integre  
r projet credit caractéristique simulation', 'intégrer projet financement caractéristique']
```

Ce découpage contribue également à augmenter le nombre d'exemples disponibles pour l'entraînement puisque à partir d'une phrase on va générer plusieurs chunks de taille différente.

Le modèle d'IA [C11, C12]

Vectorisation des textes

Une fois les données préparées, les chunks doivent être **vectorisés** afin d'être exploitables par un algorithme de machine learning. Nous utilisons ici la méthode **TF-IDF** (Term Frequency – Inverse Document Frequency) de **scikit-learn**.

Cette méthode est basée sur la fréquence des mots dans les chunks, elle permet de représenter chaque chunk sous la forme d'un vecteur numérique qui reflète l'importance relative des mots du chunk par rapport à l'ensemble des chunks. Les mots fréquents dans un chunk mais rares dans le corpus complet de chunks obtiennent ainsi un poids plus important, ce qui permet de mieux distinguer les termes caractéristiques d'un domaine spécifique.

Pour cela on crée un objet `TfidfVectorizer()` puis on l'entraîne sur notre ensemble de textes (chunks).

```
vectorizer = TfidfVectorizer()
tfidf_texts_matrix = vectorizer.fit_transform(df_data.Texts)
print("X.shape:", tfidf_texts_matrix.shape)
```

Cette opération nous fournit :

- Le vectorizer qui servira à encoder les nouvelles phrases,
- La vectorization des textes déjà collectés,
- Affiche le format (20000, 6937)

Remarques :

- Le vectorizer est sauvegardé (à l'aide de la librairie pickle), en effet le même vectorizer doit être utilisé lorsqu'une nouvelle phrase doit être analysée.
- La matrice des chunks vectorisés servira à l'entraînement du modèle d'IA.
- Le format `X.shape: (20000, 6937)` de la matrice des chunks vectorisés signifie qu'il y a 20000 chunks dans nos données et que chaque chunk est vectorisé en un vecteur de longueur 6937 ce qui est la taille totale du vocabulaire de nos documents (après suppression des stop-words et lemmatization).

Annoter les documents

Lors de la collecte des documents puis du découpage en chunks, nous attribuons à chaque chunk un « label » **0 ou 1** selon que le document auquel appartient le chunk concerne ou non le domaine des assurances.

A la fin de notre traitement nous disposons donc de **données annotées** pouvant être utilisées pour l'entraînement d'un modèle, plus précisément nous disposons d'une matrice « **TF-IDF** » de nos documents vectorisés ainsi que d'un vecteur « **Label** » de même longueur que la matrice TF-IDF et contenant uniquement des 0 ou 1 pour chacune des 2 classes : **Assurances** ou **Non Assurances**.

Modèle : régression logistique

Nous avons choisi d'entraîner une régression logistique, c'est un modèle d'apprentissage supervisé largement utilisé dans les tâches de **classification binaire**. Dans notre cadre de la **classification documentaire**, la régression logistique sera utilisée pour déterminer si un texte concerne ou non le domaine des assurances.

A partir des chunks annotés, le modèle de régression logistique apprend à associer certaines caractéristiques lexicales à la probabilité qu'un document relève du domaine des assurances. Lors de la phase de prédiction, confronté à un nouveau document, le modèle calcule un score de probabilité compris entre 0 et 1 : si cette probabilité dépasse le seuil de 0,5, le document est classé comme appartenant à la catégorie « **Assurance** », sinon il est considéré comme n'y appartenant pas.

Les avantages de cette approche sont sa simplicité d'implémentation, sa rapidité d'entraînement et son efficacité.

Séparation des données en données d'entraînement et en données de tests

Nous séparons nos données annotées en **données d'entraînement** qui serviront exclusivement à entraîner la régression logistique et en **données de tests** qui serviront exclusivement à évaluer la régression logistique.

```
X_train, X_test, y_train, y_test = train_test_split(
    tfidf_texts_matrix,
    df_data.Labels,
    test_size=0.3,
    random_state=42)
```

Il est fondamental de faire cette séparation car on n'évalue jamais un modèle sur les données ayant servi à l'entraînement.

Entraînement du modèle

On entraîne notre modèle avec les données d'apprentissage :

X : les chunks de textes

y : les labels associés à chaque chunk

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

Les métriques d'un modèle de classification

Pour un problème de classification, les métriques classiques sont :

L'Accuracy : Ratio de prédictions correctes

$$\text{Accuracy} = (TP + TN) / (TP + FP + TN + FN)$$

Ce nombre est le rapport entre les instances correctement classées (positives et négatives) de toutes les décisions.

La précision : Ratio de parité prédictive

$$\text{Précision} = (TP) / (TP + FP)$$

Ce nombre décrit le rapport des échantillons qui ont été correctement classés comme positifs parmi toutes les prédictions positives.

Le Rappel : Ratio d'égalité des chances calculé à partir du taux de vrais positifs

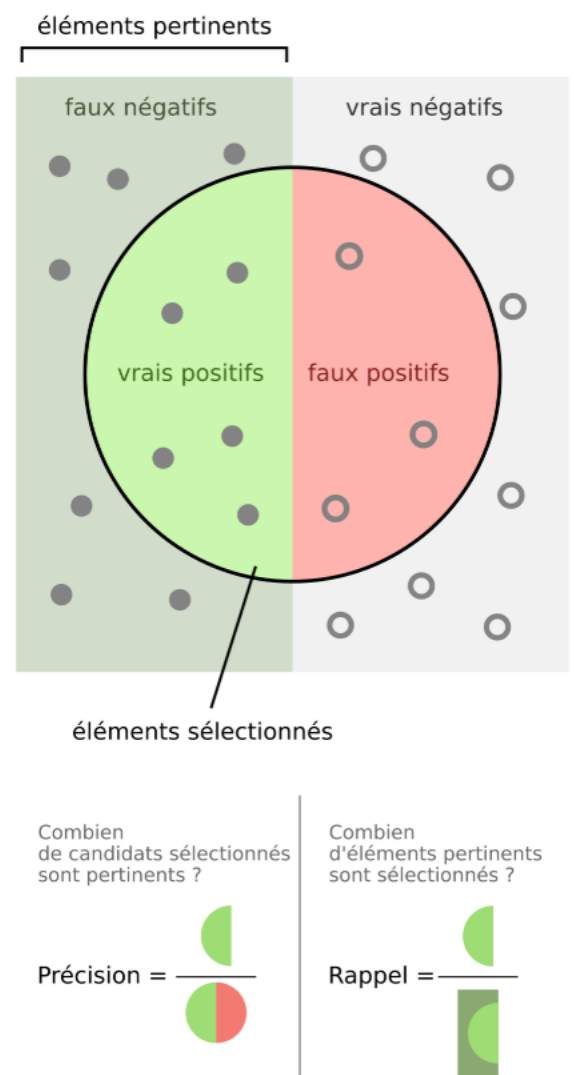
$$\text{Rappel} = TP / P$$

Ce nombre décrit les proportions d'instances positives correctement classées.

Remarque :

Le **rappel** est la capacité de notre modèle à **détecter les cas positifs dans l'échantillon proposé**, c'est-à-dire à ne pas laisser de côté des cas positifs.

Une remarque naïve serait de penser qu'il suffit de toujours prédire « positif », en effet on obtient ainsi une valeur de rappel de 1 (valeur maximale) mais l'on va annoncer des éléments négatifs comme étant positifs et dégrader fortement notre score de précision.



Evaluation de notre modèle

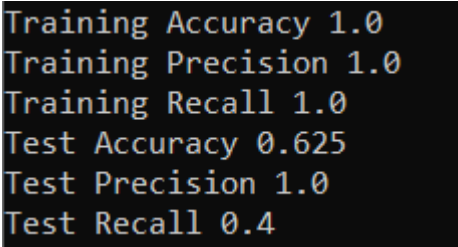
Nous **évaluons notre modèle sur les données de test**, c'est-à-dire sur l'échantillon de données qui n'a pas servi à l'entraînement, pour cela nous faisons des prédictions sur l'ensemble de test et nous comparons ces prédictions aux labels réels.

```
y_pred = model.predict(X_test) # Prédiction de notre modèle
print("Test Accuracy", accuracy_score(y_test, y_pred))
print("Test Precision", precision_score(y_test, y_pred))
print("Test Recall", recall_score(y_test, y_pred))
```

Exemples :

Un premier exemple de métriques obtenues à partir d'un faible échantillon de données.

La précision est de 1 ce qui est très bon mais le **rappel est très bas (0.4)** cela signifie que notre modèle n'a pas une bonne capacité à détecter les cas positifs dans la population mais que toutes les prédictions positives faites sont justes. Conclusion le modèle ne se trompe jamais dans ses prédictions positives mais est loin de trouver tous les cas positifs dans la population étudiée.



```
Training Accuracy 1.0
Training Precision 1.0
Training Recall 1.0
Test Accuracy 0.625
Test Precision 1.0
Test Recall 0.4
```

Selon les cas d'usage on peut préférer la précision au rappel.

Exemple 1 : Détection de fraudes

Si un inspecteur à un quota de fraudes à détecter par an, on ne lui demande pas de détecter tous les fraudeurs mais on lui impose de ne jamais se tromper lorsqu'il annonce une fraude, on privilégie dans ce cas un modèle avec **une excellente précision** et on s'autorise un **rappel plus faible**.

Exemple 2 : Détection de collisions dans le transport maritime

A l'inverse lorsque l'on détecte des collisions de bateaux, on privilégiera **un modèle qui détecte toutes les collisions**, quitte à se tromper occasionnellement et annoncer une collision lorsqu'il n'y a pas lieu, en effet annoncer une fausse collision aura pour impact de faire ralentir quelques bateaux mais n'aura pas de conséquences majeures sur le transport maritime alors que ne pas détecter une collision peut avoir des conséquences catastrophiques. Dans ce cas on privilégiera un modèle avec **un excellent rappel** et on autorisera **une précision plus faible**.

Dans notre cas de classification de textes d'assurance les conséquences sont nulles, nous essaierons de trouver un modèle avec à la fois une excellente précision et un excellent rappel. Cette tâche demande de faire un grand nombre de tests d'entraînement de modèles en variant les données utilisées, le nombre de chunks etc ...

Pour un suivi efficace de nos différents entraînements nous allons utiliser une approche MLOPS.

Monitoring du modèle avec MLFlow [C11]

MLFlow est une plateforme open source pensée pour **structurer et automatiser tout le cycle de vie du machine learning** : de l'expérimentation au déploiement.

Elle s'inscrit pleinement dans les principes du **MLOps** : la discipline qui vise à appliquer les bonnes pratiques du DevOps au monde du Machine Learning afin de rendre les modèles reproductibles, traçables, gouvernables et réellement exploitables en production.

Elle répond à un problème simple : le Machine Learning produit énormément de variables (paramètres, code, données, modèles, environnements...) et sans cadre, tout devient rapidement difficile à suivre, à reproduire ou à industrialiser.

Après avoir installé MLFlow nous démarrons le service avec la commande :

```
mlflow server --host 127.0.0.1 --port 8080
```

Puis nous encapsulons notre code d'évaluation du modèle à l'aide de commandes telles que :

```
with mlflow.start_run():
    model = LogisticRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_train)
    mlflow.log_metric("Training Accuracy", accuracy_score(y_train, y_pred))
    mlflow.log_metric("Test Precision", precision_score(y_test, y_pred))
    mlflow.log_metric("Test Recall", recall_score(y_test, y_pred))
```

On enregistre également dans MLFlow les paramètres utilisés pour chaque entraînement :

```
mlflow.set_tag("Nb documents", nb_max_docs)
mlflow.set_tag("Nb Chunks", nb_max_chunks)
```

Chaque run MLFlow va enregistrer les 3 métriques Accuracy, Précision, Rappel, on peut ensuite tracer des histogrammes pour rechercher les meilleurs modèles et regarder quels sont les paramètres utilisés.

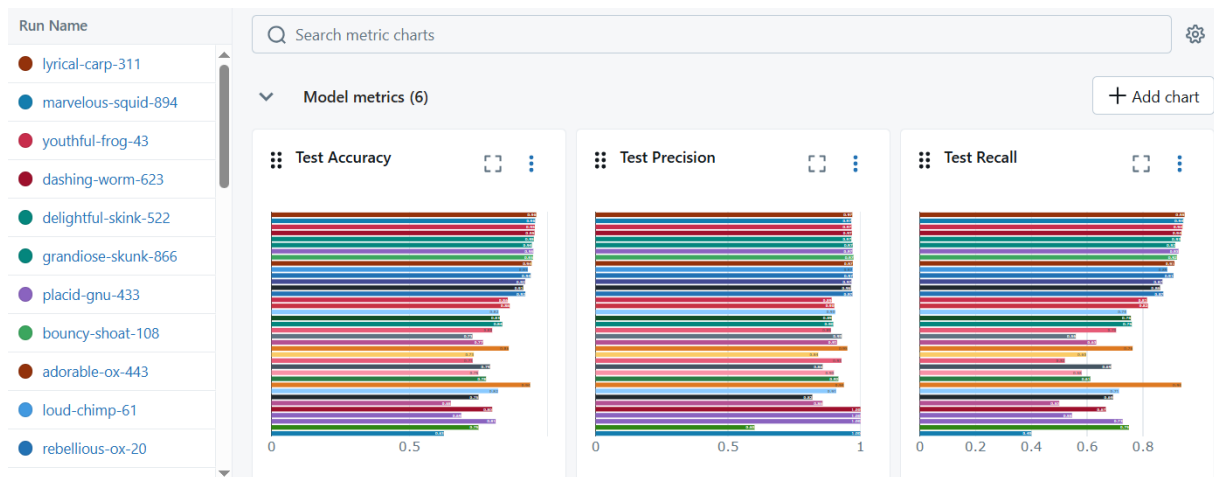


Figure 2 - Track MLFlow

Les meilleurs scores obtenus nous donnent un modèle de régression logistique avec 3 scores Accuracy, Précision, Rappel, proches de 95%.

```
Training Accuracy 0.97225
Training Precision 0.9792633410672854
Training Recall 0.9649210545116811
Test Accuracy 0.9595833333333333
Test Precision 0.969377339231031
Test Recall 0.949192070631351
```

Tests automatisés d'un modèle d'IA [C12]

Nous allons utiliser la librairie **pytest** pour réaliser des tests unitaires automatisés de notre modèle d'IA.

Un **test unitaire** vérifie le bon fonctionnement d'une unité de code isolée, en général une fonction ou une méthode. Cela permet de s'assurer que chaque bloc de code fait exactement ce qu'on attend.

Nous installons le module python pytest puis nous créons un fichier « test_main.py » qui contiendra nos tests.

Test 1 : Vérifier que le modèle est bien présent dans le fichier

```
# S'assurer que le fichier avec le modèle existe bien
def test_model_file():
    print("\n***** Test Model File")
    try:
        model_path = PATH + "reglog_ia_texts.pkl"
        assert Path(model_path).exists()
    except:
        assert False, f"Le fichier {model_path} est introuvable."
```

Test 2 : Vérifier la compatibilité entre la dimension de sortie de notre vectorizer (notre encodeur de textes) et la dimension d'entrée de notre modèle classifieur.

```
def test_vectorizer_output_dimension():
    vectorizer_path = PATH + "vectorizer_ia_texts.pkl"
    vectorizer = Load_Pickle(vectorizer_path)
    model_path = PATH + "reglog_ia_texts.pkl"
    model = Load_Pickle(model_path)
    X_transformed = vectorizer.transform(["Simple texte pour faire mon test"])
    print("TF-IDF output:", X_transformed.shape)
    print("Classifier input:", model.coef_.shape[1])
    assert X_transformed.shape[1] == model.coef_.shape[1], \
        f"ATTENTION !!!\n {X_transformed.shape[1]} != {model.coef_.shape[1]}"
```

Nous pouvons lancer les tests **pytest** manuellement :

```
(env_ca) C:\Users\Utilisateur\Documents\Projet_CA>pytest test_main.py
===== test session starts
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Utilisateur\Documents\Projet_CA
plugins: anyio-4.12.1
collected 5 items

test_main.py .....

===== 5 passed in 14.74s =====
```

Pour l'automatisation de ces tests nous utilisons **GitHub Action**, nous créons un fichier **main.yml** que l'on met dans le répertoire **.github/workflows** de notre projet.

Ce fichier **main.yml** ([voir annexes](#)) définit :

- L'évènement déclencheur pour effectuer les actions (git push),
- La commande à exécuter **pytest -vv** lors de l'évènement

Désormais lorsque **GitHub** détectera une modification de code suite à une commande git push, les actions de **Github Action** se lanceront et nous pourrons suivre l'avancée de nos tests :

The screenshot shows the 'All workflows' page on GitHub. It displays a list of workflow runs for the 'main' branch. The first run, 'Uncomment all tests for Github Action', is in a 'Queued' state. The second run, 'modified gitignore and add folders to be tracked', is completed with a green checkmark. The third run, 'update gitignore', is in a failed state with a red 'x' icon. The interface includes filters for Event, Status, Branch, and Actor, and a search bar for workflow runs.

Workflow Name	Status	Branch	Actor	Time
Uncomment all tests for Github Action	Queued	main	now	1m 5s
modified gitignore and add folders to be tracked	Completed	main	4 minutes ago	1m 5s
update gitignore	Failed	main	5 minutes ago	1m 7s

Architecture d'API [C9, C10]

Afin de rendre le modèle de classification binaire exploitable par des utilisateurs ou d'autres applications, une architecture basée sur deux **services web** a été mise en place. Cette architecture repose sur deux frameworks Python complémentaires : **FastAPI pour exposer le modèle d'intelligence artificielle** et **Flask pour fournir une interface Web utilisateur** simple permettant d'interagir avec ce modèle.

L'architecture globale repose sur une séparation claire entre la **logique d'intelligence artificielle** et la **logique d'interface utilisateur**, ce qui correspond à une architecture classique de type **backend / frontend découplée**.

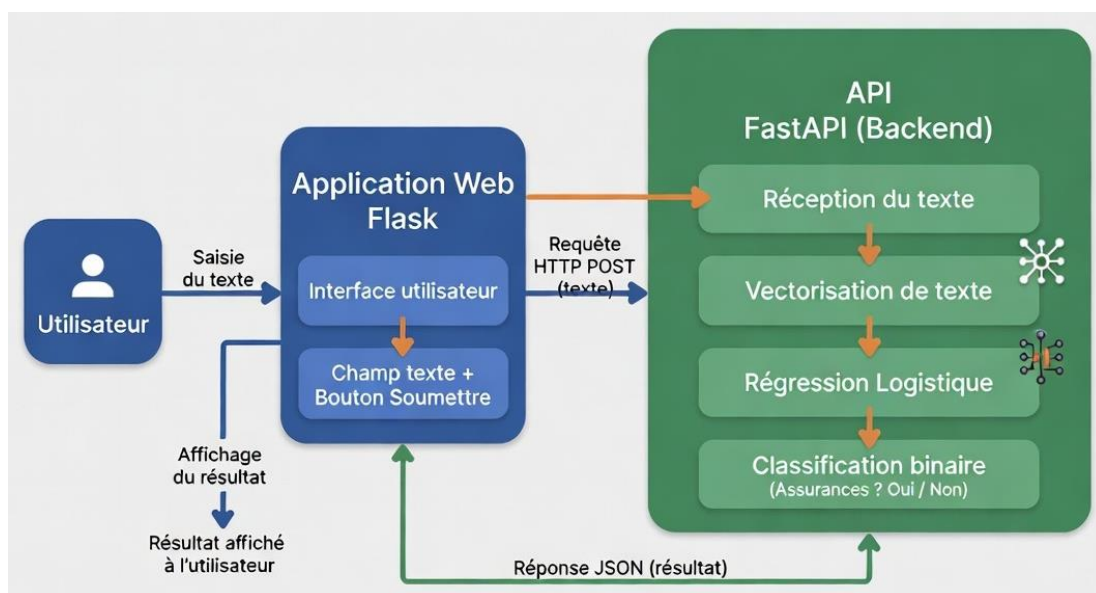


Figure 3 - Architecture Application Flask / FastAPI

Intérêts d'une telle architecture

1. La logique d'intelligence artificielle est isolée de l'interface utilisateur. Cela permet indépendamment d'améliorer le modèle d'IA ou bien de modifier l'interface utilisateur.
2. Une fois déployé, le service FastAPI peut être utilisé par plusieurs applications en même temps (application web, scripts automatisés etc...)

API FastAPI exposant le modèle d'IA [C9]

Le modèle de classification binaire entraîné (basé sur une régression logistique) a été exposé sous la forme d'une **API REST** grâce au Framework **FastAPI**.

Cette API constitue un « programme » dont l'unique rôle est de recevoir un texte, de le transformer en représentation vectorielle, puis de retourner la probabilité que ce texte

appartienne au domaine de l'assurance, cela constitue une **couche de service dédiée au modèle d'IA**, accessible via des requêtes HTTP.

Lorsqu'une requête est envoyée à l'endpoint / predict, l'API réalise les étapes suivantes :

1. Réception du texte fourni par l'utilisateur,
2. Nettoyage et vectorisation du texte **en utilisant le vectoriser** déjà entraîné pour l'ensemble des documents d'entraînement,
3. Prédiction de la probabilité d'appartenance au domaine des « Assurances » à l'aide de la **régression logistique**,
4. Retour d'une réponse JSON contenant la probabilité et la décision binaire,

FastAPI a été choisi pour plusieurs raisons :

- Très bonnes performances,
- Génération automatique **d'une documentation interactive**,
- Grande popularité pour les services de machine learning.

Règles d'authentification

Nous ajoutons un système d'authentification pour que notre API ne soit accessible qu'aux utilisateurs possédant un mot de passe. Nous utilisons le système de Tokenisation **JWT** (JSON Web Token), c'est un standard pour communiquer des données sécurisées à travers des objets JSON (données signées numériquement). Fonctionnement de JWT :

1. Le client (navigateur) envoie au serveur les informations d'authentification (password),
2. Si l'authentification est acceptée, le serveur génère un jeton JWT et le transmet au client,
3. Le client peut envoyer des requêtes pour accéder à des ressources, il y joint le token,
4. Si le token est bon, le serveur envoie au client les ressources demandées.

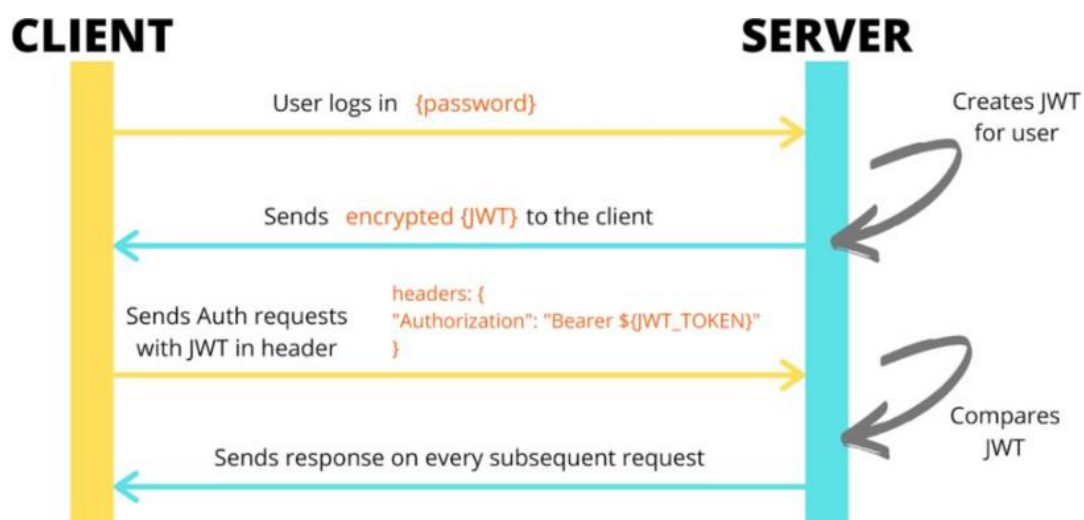


Figure 4 - Authentification JWT

Exemples d'utilisation de l'API

On lance l'API avec la commande :

```
uvicorn main:app --reload
```

Uvicorn est le nom du module python qui simule un serveur pour déployer notre application, **main** est le nom du fichier python qui contient le code de l'API et **app** est le nom de l'objet FastAPI défini dans le fichier main.py

On accède à notre API via l'URL : **http://127.0.0.1:8000/**
et la documentation interactive de l'API est disponible à l'URL :
http://127.0.0.1:8000/docs

POST /predict Predict

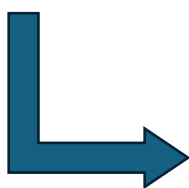
Route réalisant la prédiction à partir d'un texte saisi par l'utilisateur.

Return: Un dictionnaire contenant le texte, la probabilité d'être un texte d'assurance, et une classification binaire.

Parameters

Name	Description
text * required string (query)	recette du gâteau au chocolat

Execute



Server response	
Code	Details
200	Response body<pre>{ "text": "recette du gâteau au chocolat", "assurance_probability": 0.1202077627708864, "no_assurance_probability": 0.8797922372291136, "is_assurance": false }</pre>

API Flask exposant l'application [C10]

Afin de permettre à un utilisateur d'interagir facilement avec le modèle, une application a été développée avec **Flask**. Cette application fournit une interface simple dans laquelle l'utilisateur peut saisir un mot ou une phrase.

Le fonctionnement est le suivant :

1. L'utilisateur saisit un texte dans l'interface web,
2. L'application Flask récupère cette saisie,
3. Flask envoie une requête HTTP à l'API FastAPI du modèle d'IA,
4. l'API renvoie la probabilité de classification,
5. Flask affiche le résultat à l'utilisateur.

Flask joue donc le rôle de **client de l'API du modèle**. Cette séparation permet de garder l'application web simple tout en déléguant le traitement intelligent au service dédié.

Utilisation de l'API Flask

On lance l'API Flask comme un simple script python avec la commande :

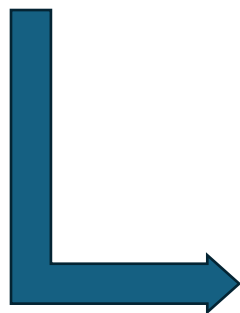
```
Python app.py
```

On accède à l'API via l'URL : **http://127.0.0.1:5000**

Classification Assurance

Quelle est la durée de prise d'effet d'une résiliation ?

Analyser



Classification Assurance

Entrez un mot ou une phrase...

Analyser

Résultat

Probabilité : 0.9642910309777881

Ce texte concerne probablement l'assurance.

Déploiement sur Render [C13]

Render est un service permettant de déployer des applications Web.

Sur Render on crée et on configure un **Web Service** que l'on relie à notre dépôt GitHub

The screenshot shows the Render dashboard for a service named 'Binary_Classification'. The left sidebar contains navigation links: Dashboard, Binary_Classification, Events, Settings, MONITOR (Logs, Metrics), and MANAGE. The main content area shows the service details: 'WEB SERVICE Binary_Classification' with 'Python 3' and 'Free' tags, and a link to 'Upgrade your instance'. Below this, the 'Service ID' is 'srv-d6t6qfndiees73cp1o20', and the 'Repository' is 'Franck-LF / Projet_CA' with the 'main' branch. The 'Public URL' is 'https://binary-classification.onrender.com'. A warning message states: 'Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.' The deployment status is 'March 18, 2026 at 10:48 AM' with a 'Live' status.

Commande à lancer lors du déploiement

Et déclencheur du déploiement (lors d'un commit sur GitHub)

Start Command

Render runs this command to start your app with each deploy.

```
$ gunicorn -k uvicorn.workers.UvicornWorker main:app --bind 0.0.0.0...
```

Edit

Auto-Deploy

By default, Render automatically deploys your service whenever you update its code or configuration. Disable to handle deploys manually.

On Commit

Edit

Une fois les paramètres validés, un déploiement se lance :

```
10:53:46 AM [q2m4g] 185.221.88.186:0 - "GET / HTTP/1.1" 200
10:53:46 AM ==> Your service is live 🎉
10:53:46 AM ==>
10:53:46 AM [q2m4g] 185.221.88.186:0 - "GET /favicon.ico HTTP/1.1" 404
10:53:46 AM ==> //////////////////////////////////////
10:53:46 AM ==>
10:53:46 AM ==> Available at your primary URL https://binary-classification.onrender.com
10:53:46 AM ==>
10:53:46 AM ==> //////////////////////////////////////
```

Une fois déployée, l'application est disponible via une URL publique :

binary-classification.onrender.com/docs#/default/predict_predict_post

text * required
string
(query)

recette tarte au chocolat

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://binary-classification.onrender.com/predict?text=recette%20tarte%20au%20chocolat' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer admin' \
  -d ''
```

Request URL

https://binary-classification.onrender.com/predict?text=recette%20tarte%20au%20chocolat

Server response

Code	Details
200	Response body <pre>{ "text": "recette tarte au chocolat", "assurance_probability": 0, "is_assurance": false }</pre>

Active (1) Suspended (3) All (4)

Q Search services

SERVICE NAME 4	STATUS	RUNTIME	REGION	UPDATED ↓
Binary Classification	✓ Deployed	Python 3	Oregon	41min
Projet CA	Suspended by you	Python 3	Oregon	14h
test	Suspended by you	Valkey 8	Oregon	1d
MLOps	Suspended by you	Python 3	Oregon	1y

Conclusion

Ce projet de classification documentaire a permis de transformer une problématique métier — la classification automatique de documents — en une solution logicielle robuste, scalable et prête pour la production. Cela m’a amené à poser une réflexion sur l’organisation d’un projet complet

Un cycle de vie complet, de la donnée au déploiement

Le travail réalisé a suivi une démarche rigoureuse en plusieurs étapes clés :

- **Le cycle complet du traitement des données** : La collecte, nettoyage et préparation des données, vectorisation.

- **L'entraînement du modèle** : identification des variables explicatives et des variables expliquées, phase de séparation des données d'entraînement et de test puis entraînement du modèle de **Régression Logistique**.
- **L'industrialisation (MLOps)** : Au-delà du simple modèle, l'intégration de **MLFlow** assure un monitoring continu des performances, tandis que les tests automatisés garantissent la fiabilité du code à chaque évolution.
- **Une architecture moderne** : Le choix d'une architecture découplée, utilisant **FastAPI** pour l'exposition du modèle et une application cliente **Flask** pour l'interface utilisateur, offre une grande flexibilité technique.

Vers une intégration continue

Enfin, la mise en place d'un pipeline de **Livraison et Intégration Continues (CI/CD)** marque le passage d'un prototype expérimental à un produit industriel. Cette automatisation permet de déployer de nouvelles versions en toute confiance, minimisant les erreurs humaines et optimisant le temps de mise sur le marché.

Perspectives futures

Ce projet ne faisait pas partie des demandes du Crédit Agricole et a été développé comme un projet personnel mais ce projet pourrait être intégré à mon projet de moteur de recherche documentaire (voir le livrable E3) en tant que filtre de requêtes utilisateur pour nous assurer que les requêtes utilisateurs concernent bien le domaine des assurances ou non.

Annexes

Fichier YML pour les tests automatisés

```
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  tests_job:
    runs-on: ubuntu-latest
    steps:
      # Étape pour récupérer le code
      - name: Checkout repository
        uses: actions/checkout@v4

      # Étape pour installer Python et les dépendances si nécessaire
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Install dependencies
        run: pip install -r requirements.txt

      # Étape pour exécuter les tests
      - name: Run tests
        run: pytest -vv
```